

Python Functions: A Beginner's Guide

A simple introduction to what functions are, why we use them, and how to write and call them in Python.

1. What is a function?

A function is a reusable block of code that performs a specific task. Instead of writing the same code repeatedly, you put it inside a function and call the function whenever you need it.

Think of a function like a machine: you give it an input, it performs some work, and it can give you an output.

Basic structure:

```
def function_name():  
    # code goes here
```

2. Creating and calling a function

The **def** keyword tells Python that you are defining a function.

```
def greet():  
    print("Hello!")  
  
greet()
```

The first part creates the function. The line **greet()** calls it, which makes Python execute the code inside it.

3. Parameters

Parameters allow a function to receive information. They are written inside the parentheses when defining the function.

```
def greet(name):  
    print("Hello", name)  
  
greet("Victor")
```

Here, **name** is the parameter and **"Victor"** is the argument passed to the function.

4. Multiple parameters

A function can accept more than one parameter.

```
def add_numbers(a, b):  
    print(a + b)  
  
add_numbers(5, 3)
```

Output: 8

5. Returning a value

A function can send a result back to the code that called it. This is done using **return**.

```
def add_numbers(a, b):  
    return a + b  
  
result = add_numbers(5, 3)  
print(result)
```

The function calculates 5 + 3 and returns 8. The returned value is stored in **result**.

6. print() vs return

print()	return
Displays something on the screen.	Sends a value back to the caller.
Mainly used to show information.	Used when another part of the program needs the result.

For example, **return** is useful when you want to calculate something and use the answer later.

7. Default parameters

You can give a parameter a default value. If the caller does not provide a value. Python uses the default.

```
def greet(name="Guest"):  
    print("Hello", name)  
  
greet()  
greet("Victor")
```

The first call uses **Guest**. The second call uses **Victor**.

8. Why functions are useful

- **Reusability:** Write code once and use it many times.
- **Organization:** Break a large program into smaller tasks.
- **Readability:** Function names can make code easier to understand.
- **Debugging:** Smaller pieces of code are easier to test and fix.
- **Maintenance:** Changes can often be made in one place.

9. A practical example

Suppose you are building a small student system. Instead of putting everything into one large block of code, you can create separate functions.

```
def calculate_average(mark1, mark2, mark3):
    return (mark1 + mark2 + mark3) / 3

def check_pass(mark):
    return mark >= 50

average = calculate_average(70, 60, 80)
passed = check_pass(average)

print("Average:", average)
print("Passed:", passed)
```

Each function has one clear responsibility. This makes the program easier to understand and expand.

10. Function scope

Variables created inside a function are normally local to that function. They are not automatically available outside it.

```
def calculate():
    number = 10
    return number

result = calculate()
print(result)
```

The variable **number** exists inside the function, while **result** receives the value returned by the function.

11. A useful way to think about functions

When writing a program, ask: **What individual tasks does my program need to perform?** Each clear task can often become a function.

For example:

```
get_student_data()
clean_data()
calculate_average()
save_to_database()
display_results()
```

This becomes especially useful as your Python programs become larger, including data engineering and machine learning projects.

12. Beginner practice

1. Write a function called **square** that accepts a number and returns its square.
2. Write a function called **is_even** that returns True if a number is even.
3. Write a function called **maximum** that accepts two numbers and returns the larger one.
4. Write a function called **calculate_average** that accepts three numbers and returns their average.
5. Write a function called **greet** that accepts a name and prints a greeting.

Quick summary

def creates a function. **Parameters** allow a function to receive data. **Arguments** are the actual values passed to it. **return** sends a result back. Functions help make programs reusable, organized, readable, and easier to maintain.